

K-Means Parallel Benchmarking Report

Parallel and Distributed Computing Project

I220908 & I221095 PDC Project

Benchmark Implementations: Serial, Naive Parallelism, Optimized Parallelism, Scikit-Learn, and PyTorch

Target Hardware	Apple M2, 8 logical cores, 8 GB RAM
Datasets	Tier 1: 10,000 x 8, Tier 2: 100,000 x 16, Tier 3: 2,000,000 x 64
Maximum Iterations	20
Primary Metric	Execution time, speedup, throughput, memory, power, IPC

1. Project Overview

This project evaluates K-Means clustering performance across serial, parallel, optimized, and framework-based implementations. The goal is to study how execution time, throughput, CPU utilization, memory usage, power behavior, and scaling change across increasing dataset sizes and implementation styles.

The project was developed as a Parallel and Distributed Computing benchmark system. All implementations use the same datasets and the same reporting format so that results are directly comparable.

2. Objectives

- Implement a serial K-Means baseline.
- Implement a naive multi-threaded K-Means version.
- Implement a hardware-optimized K-Means version for Apple Silicon M2.
- Compare project implementations with Scikit-Learn and PyTorch framework implementations.
- Collect standardized timing, throughput, memory, CPU utilization, power, and hardware metadata.
- Save all results into reusable TXT and CSV files for report screenshots and tabular analysis.

3. Hardware and Software Environment

The benchmark system targets:

- Processor: Apple M2
- Logical cores: 8
- Memory: 8 GB unified memory
- Operating system: macOS
- Power tool: Apple `powermetrics`
- C++ compiler: Apple Clang compatible C++17 compiler
- Python environment: project-local virtual environment
- Python frameworks: pandas, scikit-learn, PyTorch

Power metrics require sudo privileges on macOS. If sudo credentials are not active or `powermetrics` cannot sample a short benchmark, the benchmark still runs and reports power fields as unavailable.

4. Dataset Description

Three synthetic CSV datasets are used:

Dataset	Tier	Points	Dimensions	Clusters
tier1_small.csv	Tier 1 (Small)	10,000	8	5
tier2_medium.csv	Tier 2 (Medium)	100,000	16	8
tier3_large.csv	Tier 3 (Large)	2,000,000	64	10

The datasets are stored in `data/` and can be regenerated using `python3 data/generate_datasets.py`.

5. Benchmarking Rules

To keep all versions comparable:

- Maximum iterations are fixed at `MAX_ITERATIONS = 20`.
- SSE / Final SSE is not calculated during benchmarking.
- Each dataset uses its predefined number of clusters.
- Serial Baseline uses 1 thread.
- Naive Parallelism and Optimized Parallelism are benchmarked with 1, 2, 4, and 8 threads.
- Scikit-Learn and PyTorch use framework-level execution behavior and report the available/default thread count.
- Every implementation writes one aggregate TXT file and one aggregate CSV file.

6. Implementations

6.1 Serial Baseline

The Serial Baseline is a C++ K-Means implementation adapted from the provided baseline source. It runs the standard K-Means loop using one thread and serves as the reference for speedup calculations.

Output files:

- `results/Serial_Baseline_Results.txt`
- `results/Serial_Baseline_Results.csv`

6.2 Naive Parallelism

The Naive Parallelism version keeps the serial algorithm structure but uses `std::thread` to parallelize point assignment and distance computation. This version is intentionally simple and is used to observe basic thread scaling.

Output files:

- `results/Naive_Parallelism_Results.txt`
- `results/Naive_Parallelism_Results.csv`

6.3 Optimized Parallelism

The Optimized Parallelism version is tuned for Apple Silicon M2. It uses:

- Flat `float32` row-major data layout
- Cache-friendly sequential memory access
- Thread-local centroid accumulation
- Reduced synchronization overhead
- Buffer reuse
- NEON SIMD distance calculation where applicable

Output files:

- `results/Optimized_Parallelism_Results.txt`
- `results/Optimized_Parallelism_Results.csv`

6.4 Scikit-Learn K-Means

The Scikit-Learn benchmark uses `sklearn.cluster.KMeans` with a fixed maximum iteration cap. This provides a high-level framework comparison against the manual C++ implementations.

Output files:

- `results/Scikit_KMeans_Results.txt`
- `results/Scikit_KMeans_Results.csv`

6.5 PyTorch K-Means

The PyTorch benchmark implements K-Means manually using tensor operations. It uses MPS acceleration on Apple Silicon when available and falls back to CPU if MPS is unavailable.

Output files:

- `results/PyTorch_KMeans_Results.txt`
- `results/PyTorch_KMeans_Results.csv`

7. Standard Metrics

Each benchmark report includes:

- Algorithm Version
- Dataset File Name

- Dataset Tier
- Dataset Dimensions
- Number of Points
- Number of Clusters
- Threads Used
- Execution Time
- Speedup
- Throughput
- Convergence Iterations
- Memory Usage
- Power Metrics Status
- CPU Power
- GPU Power
- Package Power
- Package Energy
- CPU Utilization
- Instructions/Cycle
- Hardware Information
- Status

8. Result Summary

The following summary is based on the current CSV result files in `results/`.

8.1 Serial Baseline

Dataset	Time (s)	Throughput (points/s)	Memory (MB)	IPC
Tier 1	0.002440	4,098,710	2.98	N/A
Tier 2	0.073607	1,358,572	20.84	N/A
Tier 3	11.719110	170,661	1120.45	2.4069

The Serial Baseline provides the reference timing. Runtime increases sharply from Tier 2 to Tier 3 because the large dataset has both many more points and a much higher dimensionality.

8.2 Naive Parallelism

Dataset	Threads	Time (s)	Speedup	Throughput (points/s)	Package Energy (J)
Tier 1	1	0.002835	0.8608x	3,527,959	N/A
Tier 1	2	0.002043	1.1943x	4,894,564	N/A
Tier 1	4	0.002383	1.0241x	4,197,272	N/A
Tier 1	8	0.002604	0.9370x	3,839,999	N/A
Tier 2	1	0.074158	0.9926x	1,348,481	N/A
Tier 2	2	0.049246	1.4947x	2,030,617	N/A
Tier 2	4	0.043619	1.6875x	2,292,555	N/A
Tier 2	8	0.031533	2.3343x	3,171,243	N/A
Tier 3	1	11.727125	0.9993x	170,545	61.7151
Tier 3	2	6.029182	1.9437x	331,720	55.2712
Tier 3	4	3.189790	3.6739x	627,001	45.7783
Tier 3	8	2.419892	4.8428x	826,483	39.0111

Naive Parallelism scales best on the largest dataset. The smaller Tier 1 dataset is too small to consistently benefit from threading because thread creation and synchronization overhead become significant compared with the actual computation.

8.3 Optimized Parallelism

Dataset	Threads	Time (s)	Speedup	Throughput (points/s)	Package Energy (J)
Tier 1	1	0.002416	1.0098x	4,138,573	N/A
Tier 1	2	0.001728	1.4120x	5,786,759	N/A
Tier 1	4	0.002448	0.9966x	4,084,342	N/A
Tier 1	8	0.002381	1.0250x	4,200,798	N/A
Tier 2	1	0.045008	1.6354x	2,221,835	N/A
Tier 2	2	0.022020	3.3427x	4,541,317	N/A
Tier 2	4	0.027289	2.6973x	3,664,486	N/A
Tier 2	8	0.020190	3.6458x	4,953,070	N/A
Tier 3	1	2.881433	4.0671x	694,099	22.4354
Tier 3	2	1.558096	7.5214x	1,283,618	21.3038
Tier 3	4	0.949025	12.3486x	2,107,425	22.1019
Tier 3	8	0.836018	14.0178x	2,392,294	14.2081

The optimized implementation is the strongest C++ result. The flat memory layout, reduced allocation overhead, and SIMD-aware distance computation substantially reduce runtime, especially for the large Tier 3 dataset.

8.4 Framework Results

Implementation	Dataset	Time (s)	Speedup	Throughput (points/s)	Memory (MB)
Scikit-Learn	Tier 1	0.069476	0.0351x	143,935	160.70
Scikit-Learn	Tier 2	0.041747	1.7632x	2,395,403	208.09
Scikit-Learn	Tier 3	1.606677	7.2940x	1,244,805	1897.34
PyTorch	Tier 1	0.563273	0.0043x	17,753	304.52
PyTorch	Tier 2	15.307218	0.0048x	6,533	351.95
PyTorch	Tier 3	47.704758	0.2457x	41,925	1384.08

Scikit-Learn performs well on larger datasets due to optimized framework internals. PyTorch is slower in this project because the manual tensor implementation has framework overhead and uses repeated batched operations that are not as efficient as the optimized C++ implementation for this workload.

9. Best Observed Results

Dataset	Best Version	Threads	Time (s)	Speedup
Tier 1	Optimized Parallelism	2	0.001728	1.4120x
Tier 2	Optimized Parallelism	8	0.020190	3.6458x
Tier 3	Optimized Parallelism	8	0.836018	14.0178x

The optimized implementation is the best overall performer in the available results.

10. Analysis

Small Dataset Behavior

For Tier 1, execution times are extremely small, so benchmark overhead, thread scheduling, and framework startup costs dominate. Parallel versions do not always improve performance because there is not enough work to amortize threading overhead.

Medium Dataset Behavior

For Tier 2, parallelism becomes beneficial. Both Naive Parallelism and Optimized Parallelism improve over the serial baseline, with the optimized version showing stronger gains due to better memory layout and lower overhead.

Large Dataset Behavior

Tier 3 clearly benefits from optimization. The optimized version achieves the largest speedup and highest throughput. Power metrics were also more reliable for the large dataset because the execution time was long enough for `powermetrics` to collect samples.

Power and Energy Behavior

Power sampling is most meaningful for longer runs. Very short Tier 1 and Tier 2 runs often finish before enough samples are collected. On Tier 3, optimized parallelism reduces total package energy because the runtime reduction is large enough to offset higher parallel power draw.

IPC Observations

Instructions per cycle was captured for some longer runs. The Serial and Naive Tier 3 results show IPC around 2.4 for 1 to 4 threads, while the Naive 8-thread run drops to about 1.99 IPC. This suggests that high thread counts can increase resource contention. Scikit-Learn Tier 3 reports a higher IPC, indicating that its low-level implementation keeps the processor pipelines busy during its shorter run.

11. How to Reproduce

Build C++ implementations:

```
cd /Users/salman/Desktop/I220908_PDC_Project

make -C src/Serial_Baseline

make -C src/Naive_Parallelism

make -C src/Parallelism
```

Run all benchmarks:

```
cd /Users/salman/Desktop/I220908_PDC_Project

sudo -v

ENABLE_POWER_METRICS=1 src/Serial_Baseline/run_serial_benchmarks.sh

ENABLE_POWER_METRICS=1 src/Naive_Parallelism/run_naive_parallel_benchmarks.sh

ENABLE_POWER_METRICS=1 src/Parallelism/run_optimized_parallel_benchmarks.sh

ENABLE_POWER_METRICS=1 src/Frameworks/run_framework_benchmarks.sh
```

12. Conclusion

The benchmark results show that implementation strategy has a major impact on K-Means performance. Basic parallelism improves performance on larger datasets, but the optimized Apple Silicon implementation provides the best overall result. It reduces Tier 3 runtime from 11.719110 seconds in the Serial Baseline to 0.836018 seconds with 8 threads, achieving a speedup of 14.0178x.

The project demonstrates that effective parallel performance depends not only on using more threads, but also on memory layout, synchronization overhead, SIMD/vectorization, and workload size. For small datasets, overhead can dominate and reduce the benefit of parallel execution. For large datasets, the optimized parallel implementation delivers substantial improvements in both runtime and energy efficiency.